

TECNOLOGÍAS DE LA INFORMACIÓN
EN LINEA
INGENIERÍA DE SOFTWARE

3 créditos



Profesor Autor:

Ing. Freddy Malla García, Mtr.

Titulaciones	Semestre
INGENIERÍA DE SOFTWARE	SEXTO

Tutorías: El profesor asignado publicará en el entorno virtual de aprendizaje online.utm.edu.ec, y sus horarios de conferencias se indicarán en la sección **CRONOGRAMA**.

PERIODO ABRIL - AGOSTO 2026



Contenido

Unidad II Modelado de requerimientos desde el enfoque análisis estructurado	1
1. Ingeniería de requerimientos.....	1
1.1. Requerimientos funcionales y no funcionales.	5
1.1.1. Requerimientos funcionales	5
1.1.2. Requerimientos no funcionales.....	7
1.1.3. Requerimientos del dominio	9
1.2. Requerimientos del usuario.....	10
1.3. Requerimientos del sistema.....	11
1.4. Especificación de la interfaz.....	13
1.5. El documento de requerimientos del software.....	14
1.6. Procesos de la ingeniería de requisitos.....	16
2. Modelados.	18
2.1. Modelado orientado al flujo	19
2.1.1. Creación de un modelo de flujo de datos	19
2.1.2. Creación de un modelo de flujo de control.....	23
2.1.3. La especificación de control	24
2.1.4. La especificación del proceso	26
2.2. Modelo de flujo de datos.....	27
2.3. Modelado de datos	29
REFERENCIA BIBLIOGRÁFICA	33

Índice de Ilustración

Ilustración 1: Tipos de requerimientos no funcionales.	8
Ilustración 2: Notaciones para la especificación de requerimientos	13
Ilustración 3: El proceso de ingeniería de requerimientos	17
Ilustración 4: Modelo en espiral de los procesos de la ingeniería de requerimientos	18
Ilustración 5: DFD en el nivel de contexto para la función de seguridad de CasaSegura .	20
Ilustración 6: DFD de nivel 2 que mejora el proceso vigilar sensores.....	22
Ilustración 7: DFD de nivel 1 para la función de seguridad de CasaSegura	23
Ilustración 8: Diagrama de estado para la función de seguridad de CasaSegura	25
Ilustración 9: Activación del proceso para la función de seguridad de CasaSegura	26
Ilustración 10: Diagrama de flujo de datos del procesamiento de un pedido.	28
Ilustración 11: Modelo semántico de datos para el sistema L1BSYS.....	30
Ilustración 12: Ejemplos de entradas de diccionario de datos.	32

Índice de tabla

Tabla 1: Tareas diferentes de la ingeniería de requerimientos	2
---	---

Unidad II Modelado de requerimientos desde el enfoque análisis estructurado

1. Ingeniería de requerimientos

Diseñar y construir programas de computadora es simplemente desafiante, creativo y divertido. De hecho, la creación de software es tan buena que muchos desarrolladores de software quieren avanzar justo antes de tener una comprensión clara de lo que se necesita.

Asumen que todo se aclarará a medida que se desarrolle, como participantes del proyecto solo podrán comprender sus necesidades después de estudiar las primeras iteraciones del programa, por lo que las cosas cambian tan rápido que cualquier tentativa de comprender los requisitos es una pérdida de tiempo, que se obtienen ganancias al producir un programa que funcione. ¿Qué hace que estos argumentos sean así? La magia es que tienen elementos de verdad, pero todos están equivocados y pueden conducir a un proyecto fallido (Pressman, 2010).

Se denomina ingeniería de requerimientos a una serie de tareas y técnicas que conducen a la comprensión de los requisitos, desde el punto de vista de los procesos del software, la ingeniería de requisitos es uno de los procedimientos importantes de la ingeniería del software que inicia a partir de la actividad de comunicación y seguir modelando. Debe adaptarse a las necesidades del proceso, proyecto, producto y personas que realizan el trabajo. Los requisitos técnicos cierran la brecha entre el diseño y la construcción. Pero, ¿De dónde viene el puente? Se podría decir que comienza en los pies de los participantes, en el proyecto (por ejemplo, gerentes, clientes, usuarios) aquí se identifican las necesidades del proyecto, se describen casos de uso, funciones, características y se definen las restricciones del proyecto. Otros pueden sugerir que comienza con una definición más grandes del sistema, donde el software es solo un componente del alcance del sistema más grande. Pero no importa dónde comience, el viaje a través del puente lo llevará allí a la parte superior del proyecto, lo que le permite comprobar el

contexto de cómo funciona el programa, que debe hacerse, las necesidades especiales que deben satisfacerse mediante el diseño y la construcción; las prioridades dirigen el orden en que se lleva a cabo el trabajo, así como la información, los roles y los comportamientos que tendrán un profundo impacto en los resultados del diseño (Pressman, 2010).

La ingeniería de requerimientos proporciona los requisitos automatizados apropiados para comprender qué es lo que el cliente quiere, el análisis de necesidades, evaluación de viabilidad, negociación de una solución razonable, definición inequívoca de la solución, confirmación de especificaciones y gestión de requisitos a medida que se entregan a un sistema en funcionamiento (Pressman, 2010).

Incluye siete tareas: concepción, indagación, elaboración, negociación, especificación, validación y administración. Es fundamental tener en cuenta que algunas de estas tareas se ejecutan en paralelo y todas ellas se ajustan a las necesidades del proyecto (Pressman, 2010).

Tabla 1: Tareas diferentes de la ingeniería de requerimientos

Tareas diferentes de la ingeniería de requerimientos	
Concepción	La mayor parte de proyectos empieza cuando se reconoce una necesidad del negocio o se descubre un nuevo mercado o servicio potencial. En el diseño del proyecto, se determina la comprensión básica del problema, quién quiere una solución, la naturaleza de la solución requerida, la eficacia de la comunicación inicial y la colaboración entre los demás participantes y la parte del equipo y el equipo de software.
Indagación	Problemas encontrados cuando ocurre una indagación. Se identifican cierto número de problemas que se encuentran cuando ocurre la indagación: - Problema de alcance: el límite del sistema es incorrecto Identificación del cliente o usuario final con especificación de los datos Técnicos inútiles confundidos. - Problemas de entendimiento: el cliente o el usuario no está seguros de lo que necesitan, se equivocan, captan mal las capacidades y limitaciones de su entorno de TI, ignorar la información que dan por sentado,

	identifican los requisitos no coinciden con la solicitud clientes u otros usuarios, realizar pedidos ambiguos. - Problemas de volatilidad: los requisitos varían según el tiempo
Elaboración	La información conseguida del cliente durante el proceso de diseño se amplía y refina durante el desarrollo. Esta misión se enfoca en desplegar un modelo refinado de los requisitos que definen diferentes aspectos del software, su comportamiento e información. El desarrollo está impulsado por la creación y mejora de escenarios de usuario que detallan cómo los usuarios finales (y otras partes interesadas) interactúan con el sistema. Cada escenario se presenta al usuario con la sintaxis adecuada para extraer las clases de análisis, que son entidades de las que el campo de negocio es perceptible para el usuario final. Se especifican las características de cada clase de análisis y servicios requeridos por cada uno de ellos. Se crean relaciones y colaboraciones entre categorías y algunos diagramas adicionales.
Negociación	No es raro que los clientes y usuarios exijan más de lo que pueden lograr. Recursos comerciales limitados. Los clientes, usuarios y otros participantes ordenan sus requerimientos por orden de prioridad y después analicen sus conflictos. Estos conflictos deben armonizarse mediante un proceso de negociación. Se alienta a los clientes, usuarios y otras partes interesadas a priorizar sus necesidades y luego analizar los conflictos. Usa tu método de repetición preferido, se evalúan los requisitos, sus costos y riesgos y se resuelven los conflictos internos; se eliminan, combinan o modifican algunos requisitos para que cada parte cumpla con un nivel de satisfacción.
Especificación	En el contexto de los sistemas informáticos (y software), el término especificación tiene diferentes significados para diferentes personas. Puede ser un documento escrito, un conjunto de modelos gráficos, un modelo matemático formal, un conjunto de casos de uso, un prototipo o cualquier combinación de ellos. Algunos sugieren que para la especificación se desarrolle y utilice una "plantilla estándar" [Som97], ya que conduce a los requisitos mostrados de manera coherente y por lo tanto fácil de entender. Sin embargo, a veces es necesario la flexibilidad en términos de especificaciones. Para sistemas grandes, el mejor enfoque posible es un

	documento escrito que combina descripciones en lenguaje natural y modelos gráficos; para productos o sistemas pequeños ubicados en entornos bien conocidos, los casos de uso pueden ser suficientes.
Validación	Productos de trabajo de calidad creados por la tecnología de aplicación se evalúa en la etapa de validación. La validación de los requisitos analiza de la especificación para garantizar que todos estén claramente establecidos; Se descubren y corrigen inconsistencias, omisiones y errores. Los defectos y los productos funcionan de acuerdo con los procesos, proyectos y estándares de productos aplicables. El principal mecanismo para validar las solicitudes es la revisión técnica. El equipo de revisión que lo verifica incluye ingenieros de software, clientes, usuarios y otros participantes que analizan las especificaciones en busca de errores o de interpretación, aspectos que requieren aclaración, falta de información, inconsistencia (un problema grave en el diseño del producto o sistema significativos) y contradictorios o poco realistas (no asequibles)
Administración	Requisitos de los sistemas basados en los cambios en la computadora y el deseo de cambiarlos persisten a lo largo de la vida del sistema. La gestión de requisitos es un grupo de actividades que ayudan al equipo del proyecto a la identificación, seguimiento y cumplimiento de requisitos y su evolución en cualquier momento durante el desarrollo del proyecto. Muchas de estas actividades son idénticas a las técnicas de gestión de configuración de software (TAS).

Fuente: (Pressman, 2010)



Recuerde que. Recuerde que la naturaleza de la especificación variará con cada proyecto. En ciertos casos, la “especificación” no es más que un conjunto de escenarios de usuario. En otros, la especificación tal vez sea un documento que contiene escenarios, modelos y descripciones escritas.



Recuerde que. La administración formal de los requerimientos sólo se practica para proyectos grandes que tienen cientos de requerimientos identificables. Para proyectos pequeños, esta actividad tiene considerablemente menos formalidad.

1.1. Requerimientos funcionales y no funcionales.

Los requisitos del sistema de software generalmente se clasifican como funcionales y no funcionales, o como requisitos de dominio: (Sommerville, 2005)

1. Requisitos funcionales. Son declaraciones sobre qué servicios debe proporcionar el sistema, cómo debe responder el sistema a entradas específicas y cómo comportarse en situaciones específicas. En ciertos casos, los requisitos funcionales del sistema también pueden determinar qué el sistema no debería hacer (Sommerville, 2005).
2. Requisitos no funcionales. Estas son limitaciones en los servicios o funcionalidad proporcionados por el sistema. Estos contienen limitaciones de tiempo, procesos de desarrollo y estándares. Los requisitos no funcionales generalmente se aplican a todo el sistema. Por lo general, se aplica solo a funciones o servicios individuales del sistema (Sommerville, 2005).

3. Requerimientos de dominio. Estas son solicitudes que provienen del dominio de aplicación del sistema y reflejan las características y limitaciones de ese dominio. Puede ser funcional o no funcional (Sommerville, 2005).

En la práctica, la distinción entre los diversos tipos de requisitos no es tan sencilla como sugieren estas definiciones. Por ejemplo, los requisitos de seguridad del usuario pueden parecer un requisito no funcional. Sin embargo, cuando se desarrolla en detalle, puede crear otros requisitos que tengan una función clara, como la necesidad de agregar recursos del sistema para la autenticación de usuarios (Sommerville, 2005).

1.1.1. Requerimientos funcionales

Los requisitos funcionales de un sistema describen lo que se supone que debe hacer el sistema. Estos requisitos dependen del tipo de software que se esté desarrollando, de los posibles usuarios del mismo y el enfoque general de la organización al redactar los requisitos (Sommerville, 2005).

Cuando se habla en términos de requisitos del usuario, a menudo se describe en términos abstractos, pero los requisitos funcionales del sistema representan en detalle las funciones del sistema, sus entradas y salidas, excepciones. Los requisitos funcionales de un sistema de software se pueden expresar de varias maneras. Estos son algunos ejemplos de estos requisitos funcionales (Sommerville, 2005).

- Un sistema de biblioteca universitaria llamado LIBSYS para uso de estudiantes y profesores que solicitan libros y materiales de otras bibliotecas.
- El usuario debe poder buscar en el conjunto inicial de la base de datos o seleccionar un subconjunto del mismo.
- El sistema debe proporcionar un visor conveniente para que el usuario lea el documento en el almacén de documentos (Sommerville, 2005).

A cada solicitud se le debe asignar un identificador único (ID_PEDIDO), que el usuario puede copiar en el almacenamiento persistente de la cuenta. Estos requisitos funcionales del usuario definen los recursos específicos que debe proporcionar el sistema. Estos requisitos se toman de la especificación del usuario y muestra los diferentes niveles de detalle en los que se pueden escribir los requisitos funcionales (a diferencia de los requisitos 1 y 3) (Sommerville, 2005).

LIBSYS es una interfaz notable para diferentes bases de datos de artículos, aquí los usuarios pueden descargar copias de artículos de Revistas científicas y publicaciones. Imprimir según las especificaciones es la causa de muchos problemas de la ingeniería de software. Para un desarrollador de sistemas es natural dar explicaciones de requerimiento indeterminado para simplificar su implementación, pero esto no suele ser lo que el cliente quiere. Se deben implantar nuevos requisitos y hacer cambios en el sistema y esto retrasa la entrega del producto y aumenta los costos (Sommerville, 2005).

1.1.2. Requerimientos no funcionales

Los requisitos no funcionales, como su nombre indica, son requisitos que no se refiere directamente a las funciones específicas proporcionadas por el sistema, sino que se refiere a sus características sobresalientes como confiabilidad, tiempo de respuesta y capacidad de almacenar. Además, definen las limitaciones del sistema, que son las capacidades de los dispositivos de entrada/salida y las representaciones de datos utilizadas en la interfaz del sistema (Sommerville, 2005).

Los requisitos no funcionales infrecuentemente se asocian con propiedades específicas del sistema. A su vez, estos requisitos definen o restringen las características del sistema, pueden determinar el rendimiento, la seguridad, la disponibilidad y otras características destacadas del sistema. Esto expresa que a menudo son más importantes que los requisitos funcionales específicos. Los usuarios del sistema a menudo encuentran una manera de resolver el problema de la funcionalidad del sistema que realmente no satisface sus necesidades, pero el incumplimiento de los requisitos no funcionales puede implicar que todo el sistema no se puede utilizar. Por ejemplo, si el sistema de vuelo no cumple con sus requisitos de confiabilidad, no será certificado como seguro para operar; si el sistema de control en tiempo real no cumple con sus requisitos de rendimiento, las funciones de control no funcionarán correctamente (Sommerville, 2005).

Los requisitos no funcionales no solo se refieren al sistema de software que se desarrollará, algunos de estos requisitos pueden limitar el proceso que debe utilizarse para el desarrollo del sistema. Ejemplos de requisitos del proceso son las especificaciones de los estándares de calidad utilizados en el proceso, las especificaciones que el diseño debe crear con una herramienta CASE en particular y una descripción del proceso a seguir. Los requisitos no funcionales surgen de las necesidades del usuario, las restricciones presupuestarias, las políticas regulatorias, la necesidad de interoperabilidad con otros sistemas de hardware o software, o factores externos, como las leyes de confidencialidad o privacidad. La ilustración 1 es la clasificación de los requisitos no funcionales, en este diagrama, se puede ver que los

requisitos no funcionales, pueden provenir de la funcionalidad requerida del software (requisitos del producto), de la organización de desarrollo de software (requisitos organizacionales) o de fuentes externas (Sommerville, 2005).

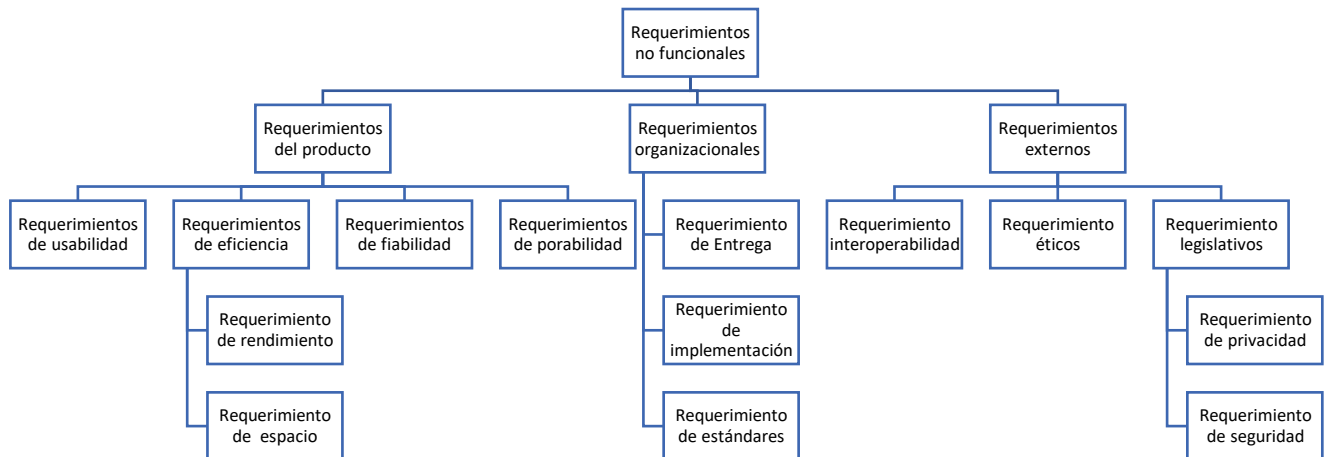


Ilustración 1: Tipos de requerimientos no funcionales.
Fuente: (Sommerville, 2005)

Los tipos de requisitos no funcionales son:

1. **Requisimientos del producto.** Estos requisitos definen el comportamiento para el producto. Algunos ejemplos son los requisitos de rendimiento de velocidad de ejecución del sistema, la cantidad de memoria requerida; los requisitos de confiabilidad definen la tasa de error del sistema como aceptable; requisitos de portabilidad y usabilidad (Sommerville, 2005).
2. **Requisimientos organizacionales.** Estos requisitos provienen de las políticas y procedimientos actuales en las organizaciones de clientes y desarrolladores. Algunos ejemplos son criterios en los que se deben utilizar procedimientos; requerimientos de implementación, como lenguajes de programación o métodos de diseño a utilizar y los requisitos de entrega que determinan cuándo se entregará el producto y su documentación (Sommerville, 2005).

3. Requerimientos externa. Esta sección cubre todos los requisitos que proceden de factores ajenos al sistema y su proceso de desarrollo. Pueden incluir requisitos de interoperabilidad que definen cómo interactúa el sistema con los sistemas de otras organizaciones; los requisitos legislativos deben seguir para asegurar que el sistema opere dentro de la ley y requisitos éticos. Se colocan en un sistema para asegurarse de que sean aceptados por los usuarios y el público (Sommerville, 2005).

1.1.3. Requerimientos del dominio

Los requerimientos de dominio se originan en el dominio de la aplicación del sistema, no de las necesidades específicas del usuario. A menudo encierran términos específicos de dominio o se refieren a conceptos de nombres de dominio. Estos pueden ser nuevos requisitos funcionales, limitaciones a los requisitos existentes o una determinación de cómo se realizarán los cálculos específicos. Los ingenieros de software a menudo encuentran esto difícil comprender cómo se relacionan con otros requisitos del sistema. Los requisitos del dominio son fundamentales porque a menudo muestran los fundamentos del dominio de la aplicación. Si no se cumplen estos requisitos, es posible que el sistema no funcione satisfactoriamente. LIBSYS tiene varios requisitos del dominio: (Sommerville, 2005)

- Se requiere una interfaz de usuario estándar para todas las bases de datos basadas en los estándares Z39.S0.
- Debido a restricciones de derechos de autor, ciertos materiales deben eliminarse inmediatamente después de su llegada. Según el requisito usuarios, estos documentos se imprimirán localmente en el servidor del sistema para que se envíen al usuario manualmente o se envíen a una impresora de red (Sommerville, 2005).

La primera condición está en diseño. Definir la interfaz de usuario para la base de datos, debe implementarse de acuerdo con un estándar de biblioteca específico. Por lo tanto, los desarrolladores deben informarse con el estándar antes de comenzar a diseñar la interfaz. El segundo reclamo se hace debido a la ley de derechos de autor (Sommerville, 2005).

Se aplica a los documentos utilizados en la biblioteca. Establece que el sistema debe incluir un recurso automático para evitar la impresión de ciertas clases de documentos. Esto representa que los usuarios del sistema bibliotecario no pueden obtener sus propias copias electrónicas de los documentos (Sommerville, 2005).

1.2. Requerimientos del usuario

Los requisitos del usuario para el sistema deben describir los requisitos funcionales y no funcionales de una manera que los usuarios del sistema puedan entender sin necesidad de conocimientos técnicos detallados. Solo necesitan determinar el comportamiento externo del sistema y evitar las características de diseño del sistema tanto como sea posible. Por lo tanto, si está escribiendo solicitudes de usuario, no debe usar términos de software, en símbolos formales o estructurados o describiendo requisitos describiendo la implementación del sistema. Debe estar escrito en un lenguaje sencillo con tablas y formularios simples y diagramas visuales intuitivos. Sin embargo, pueden presentarse algunos problemas al formar oraciones en lenguaje natural en documentos de texto: (Sommerville, 2005)

- Falta de claridad, a veces es difícil usar el lenguaje correctamente y sin hacer que el documento sea poco conciso y difícil de leer (Sommerville, 2005).
- Confusión de requerimientos, Los requisitos funcionales y no funcionales, los objetivos del sistema y la información de diseño no se distinguen claramente (Sommerville, 2005).
- Conjunción de requisitos. Varios requisitos desiguales se pueden expresar juntos en una orden (Sommerville, 2005).

Para reducir malentendidos al escribir las solicitudes de los usuarios, se debe seguir algunas pautas simples: (Sommerville, 2005)

Inventa un formato estándar y asegúrate de que se cumplan todos los requisitos. La estandarización del formato hace que las omisiones sean menos probables y las solicitudes más factibles de verificar. El formato

empleado muestra los requisitos originales en negrita, incluida una declaración de referencia con los requisitos de cada usuario y una indicación de la especificación de requisitos del sistema más detallada. También puede incluir información sobre el remitente que hizo la solicitud (la fuente del requerimiento), para que sepa a quién consultar si necesita modificar el pedido (Sommerville, 2005).

Utilizar el lenguaje de forma consistente, siempre debe diferenciar entre requisitos deseables y obligatorios. Los requisitos obligatorios son requisitos que el sistema debe dar soporte y a menudo se escribe simplemente en el futuro. Los requerimientos deseables no son fundamentales y se escribe en caso futuro (Sommerville, 2005).

Resalte el texto (negrita, cursiva o en color) para resaltar los elementos básicos de la solicitud (Sommerville, 2005).

Evitar en lo posible el uso de términos informáticos. Sin embargo, los términos técnicos detallados definitivamente se incluirán en la solicitud del usuario (Sommerville, 2005).

1.3. Requerimientos del sistema

Los requisitos del sistema son versiones extendidas de los requisitos del usuario, son empleados por los ingenieros de software como punto de partida para el diseño del sistema. Integran detalles y explican cómo el sistema debe cumplir con los requisitos del usuario (Sommerville, 2005).

Se puede utilizar como parte de un contrato de implementación del sistema, por lo tanto, debe ser una especificación completa y consistente de todo el sistema. En teoría, los requisitos del sistema solo deberían describir el comportamiento del sistema externo y sus limitaciones operativas. No deben lidiar con cómo se debe diseñar o implementar el sistema.

Sin embargo, para definir completamente un sistema de software complejo en el nivel de detalle requerido, no es práctico excluir toda la información de diseño (Sommerville, 2005).

Esto tiene varias razones:

1. Es posible que deba diseñar una arquitectura de sistema inicial para ayudar con la estructura de la especificación de requerimientos. Los requisitos del sistema se organizan de acuerdo a los diferentes subsistemas que componen el sistema (Sommerville, 2005).
2. En muchos casos, los sistemas tienen que adaptarse a los sistemas existentes. Esto restringe el diseño y estas restricciones imponen requisitos al nuevo sistema (Sommerville, 2005).
3. Necesita usar una arquitectura específica para cumplir con los requisitos no funcionales (por ejemplo, programación de n versiones para lograr fiabilidad. El regulador externo debe certificar que el sistema es seguro, puede determinar que se debe utilizar un diseño arquitectónico aprobado (Sommerville, 2005).

El lenguaje natural se usa a menudo en la escritura, además de los requisitos del usuario: las especificaciones de requisitos del sistema, pero como los requisitos del sistema son más detallados que los requisitos del usuario las especificaciones del lenguaje natural pueden ser confusas y difíciles de entender: (Sommerville, 2005).

La comprensión del lenguaje natural depende de que los escritores y lectores utilicen las mismas palabras para el mismo concepto. Esto conduce a malentendidos debido a la ambigüedad del lenguaje natural. Jackson (Jackson. 1995) dio un gran ejemplo de esto cuando habló de las señales en las escaleras mecánicas. Esto dice "se deben usar zapatos" y "se deben traer perros". Varias interpretaciones de estas frases dependen del lector (Sommerville, 2005).

La especificación de requisitos de lenguaje natural es muy flexible. Lo mismo puede decirse de forma completamente diferentes. El lector decide si los requisitos son los mismos y si son diferentes (Sommerville, 2005).

No existe una manera fácil de cambiar los requisitos del lenguaje natural. Puede ser difícil localizar todos los requisitos relevantes. Para detectar el efecto de un cambio, puede ser necesario examinar todos los requisitos en

lugar de examinar un solo conjunto de requisitos relacionados (Sommerville, 2005).

Debido a estos problemas, las especificaciones de requisitos escritas en lenguaje natural pueden ser malinterpretadas. A menudo no se descubren hasta más adelante en el proceso del software y pueden ser muy costosos de solucionar (Sommerville, 2005).

Es necesario escribir los requisitos del usuario en un lenguaje que los no expertos puedan usar y entender. Sin embargo, los requisitos del sistema se pueden escribir en notaciones más especializados, como lenguaje natural estructurado y estilizado, y modelos gráficos de requisitos, como campos adecuados para su uso en especificaciones matemáticas formales. Aquí se explica cómo se puede utilizar el lenguaje natural estructurado complementado con modelos gráficos simples para escribir los requisitos del sistema (Sommerville, 2005).

Lenguaje natural estructurado	Este enfoque depende de la definición de formularios o plantillas estándares para expresar la especificación de requerimientos.
Lenguajes de descripción de diseño	Este enfoque utiliza un lenguaje similar a uno de programación, pero con características más abstractas, para especificar los requerimientos por medio de la definición de un modelo operativo del sistema. Este enfoque no se utiliza ampliamente en la actualidad, aunque puede ser útil para especificaciones de interfaces.
Notaciones gráficas	Para definir los requerimientos funcionales del sistema, se utiliza un lenguaje gráfico, complementado con anotaciones de texto. Uno de los primeros lenguajes gráficos fue SADT (Ross, 1977) (Schoman y Ros, 1977). Actualmente, se suelen utilizar las descripciones de casos de uso (Jacobsen <i>et al.</i> , 1993) y los diagramas de secuencia (Stevens y Pooley, 1999).
Especificaciones matemáticas	Son notaciones que se basan en conceptos matemáticos como el de las máquinas de estado finito o los conjuntos. Estas especificaciones no ambiguas reducen los argumentos sobre la funcionalidad del sistema entre el cliente y el contratista. Sin embargo, la mayoría de los clientes no comprenden las especificaciones formales y son reacios a aceptarlas como un contrato del sistema.

Ilustración 2: Notaciones para la especificación de requerimientos
Fuente: (Sommerville, 2005)

1.4. Especificación de la interfaz

Casi todos los sistemas de software deben operar con otros sistemas que ya están instalados y desplegados en el entorno. Si el nuevo sistema y el sistema existente funcionan juntos, las últimas interfaces deben especificarse correctamente. Estas especificaciones deben identificarse al principio del

proceso e incluirse en la documentación (posiblemente un apéndice) de requerimientos (Sommerville, 2005).

Se pueden identificar tres tipos de interfaces:

1. Interfaces de procedimientos, en las que se encuentran los programas o subsistemas que proporcionen una variedad de servicios a los que se puede acceder a través de una llamada de procedimiento desde la interfaz. Estas interfaces a veces se denominan interfaces de programación de Aplicaciones (API) (Sommerville, 2005).
2. Estructuras de datos, que se mueven de un subsistema a otro. El modelo de datos gráficos es la mejor notación para este tipo de descripción. Si es necesario, la descripción del programa se puede generar automáticamente en Java o C++ de estas descripciones (Sommerville, 2005).

3. Representación de datos (como el orden de los bits) para un subsistema disponible ahora. Estas interfaces son muy comunes en los sistemas integrados en tiempo real. Ciertos lenguajes de programación como Ada (pero no Java) admiten este nivel de especificación. Sin embargo, la mejor manera de describirlo es seguramente usar diagrama de estructura con explicaciones que muestran la función de cada conjunto de bits (Sommerville, 2005).

1.5. El documento de requerimientos del software

Documento de requisitos de software (a veces llamado especificación de requisitos de software o SRS) es la declaración oficial de lo que debe hacer un desarrollador de sistemas. Debe incluir tanto los requisitos del usuario para el sistema como una especificación detallada de los requisitos del sistema, en algunos casos se pueden incluir dos tipos de requisitos en una misma descripción. En otras palabras, los requisitos del usuario se especifican al principio de la especificación de requisitos del sistema. Si hay una gran cantidad de requisitos, las referencias de los requisitos del sistema se pueden suministrar en un documento separado (Sommerville, 2005).

El documento de requisitos contiene una variedad de usuarios, que van desde los altos directivos de la organización que pagan por el sistema, a los ingenieros responsables de desarrollo de software. La figura 2, del libro con Gerald kottonya

sobre requisitos de ingeniería (Kontonya y Sommerville, 1998), muestra a los usuarios potenciales del documento y cómo utilizarlo (Sommerville, 2005).

La diversidad del usuario puede hacer que el documento requerido deba ser un equilibrio entre comunicar los requisitos a los clientes, definir claramente los requisitos para los desarrolladores y probadores, incluida la información sobre cómo evolucionará el sistema (Sommerville, 2005).

Cambiar información puede ayudar a los diseñadores de sistemas a evitar decisiones de diseño restrictivas y a los ingenieros responsables del mantenimiento del sistema, que deben ser modificados al sistema bajo nuevos pedidos (Sommerville, 2005).

El nivel de detalle requerido en el documento de requisitos depende del tipo de sistema que se está desarrollando y el proceso de desarrollo utilizado. Cuando el sistema se desarrolle por un contratista externo, las especificaciones de los sistemas críticos requieren ser claros y muy detallados y cuando se utiliza un proceso de desarrollo iterativo en una organización, el documento de requisitos puede ser menos detallado y cualquier ambigüedad se resuelve durante el desarrollo del sistema (Sommerville, 2005).

Varias organizaciones grandes. Como el Departamento de Defensa de EE. UU. y el IEEE establece estándares para documentos de requisitos. Davis (Davis, 1993) Analiza algunos de estos estándares y compara sus contenidos. El estándar más conocido es IEEE/ANSI 830-1998 (IEEE, 1998). Estándar IEEE recomienda la siguiente estructura del documento de pedido: (Sommerville, 2005).

1. Introducción

1.1 Propósito del documento de requerimientos

1.2 Alcance del producto

1.3 Definiciones, acrónimos y abreviaturas

1.4 Referencias

1.5 Descripción del resto del documento

2. Descripción general

2.1 Perspectiva del producto

2.2 Funciones del producto

2.3 Características del usuario

2.4 Restricciones generales

2.5 Suposiciones y dependencias

3. Requerimientos específicos: contienen los requerimientos funcionales, no funcionales y de interfaz. Obviamente, ésta es la parte más sustancial del documento, pero debido a la amplia variabilidad en la práctica organizacional, no es apropiado definir una estructura estándar para esta sección. Los requerimientos pueden documentar las interfaces externas, describir la funcionalidad y el rendimiento del sistema, especificar los requerimientos lógicos de la base de datos, las restricciones de diseño, las propiedades emergentes del sistema y las características de calidad (Sommerville, 2005).

4. Apéndices

5. Índice

Aunque el estándar IEEE no es perfecto, tiene muchas pautas sobre cómo escribir requisitos y cómo evitar problemas. Es demasiado general para ser un estándar para organización. Es un marco general que se puede transformar y adaptar para definir un estándar que se ajuste a las necesidades de una organización en particular (Sommerville, 2005).

1.6. Procesos de la ingeniería de requisitos.

El objetivo del proceso técnico es crear y mantener un documento de requisitos del sistema. El proceso general es consistente con los cuatro subprocesos de alto nivel de requerimientos técnicos. Esto se relaciona con la evaluación beneficiosa del sistema para el negocio (estudio de viabilidad); el descubrimiento de requisitos (obtención y análisis); convierta estos requisitos en formularios estándar (especificación) y la verificación de que los

requisitos especifiquen el sistema requerido que el cliente desea (validación)(Sommerville, 2005).

La ilustración 2 muestra la relación entre estas actividades, también muestra el documento que se prepara para cada paso del proceso de ingeniería de requisitos(Sommerville, 2005).

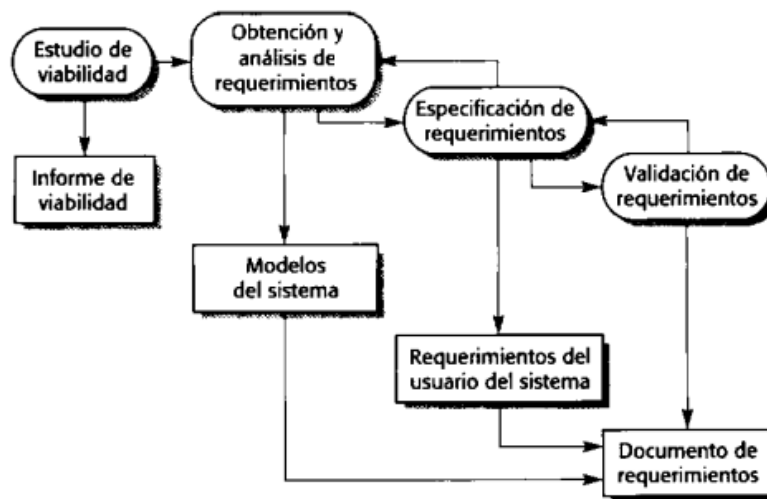


Ilustración 3: El proceso de ingeniería de requerimientos
Fuente: (Sommerville, 2005)

Las actividades que se muestran en la ilustración 2 se ocupan del descubrimiento, la documentación y la verificación de los requisitos. Pero en casi todos los sistemas los requisitos cambian. Las personas involucradas tienen una mejor comprensión de lo que quieren que haga software; la organización que compra el sistema cambia; se realizan cambios en el sistema de hardware, software y el entorno organizacional. Este proceso de gestión de cambios en los requisitos se denomina gestión de requisitos(Sommerville, 2005).

Otra vista del proceso técnico requerido se presenta en ilustración, esto indica que este proceso es una actividad de las tres etapas en las que la eliminación de activos se regula como un proceso frecuente alrededor de una espiral. La cantidad y el esfuerzo dedicado a cada actividad en una iteración depende de la etapa del proceso general y el tipo de sistema a desarrollar. Al iniciar el proceso, la mayoría de los esfuerzos se dedicarán a comprender los

requisitos de trabajo de alto nivel y los requerimientos no funcionales y del usuario. Al final del proceso, en el anillo exterior de la espiral, se dedicará más esfuerzo a los requisitos del sistema de ingeniería y modelado (Sommerville, 2005).

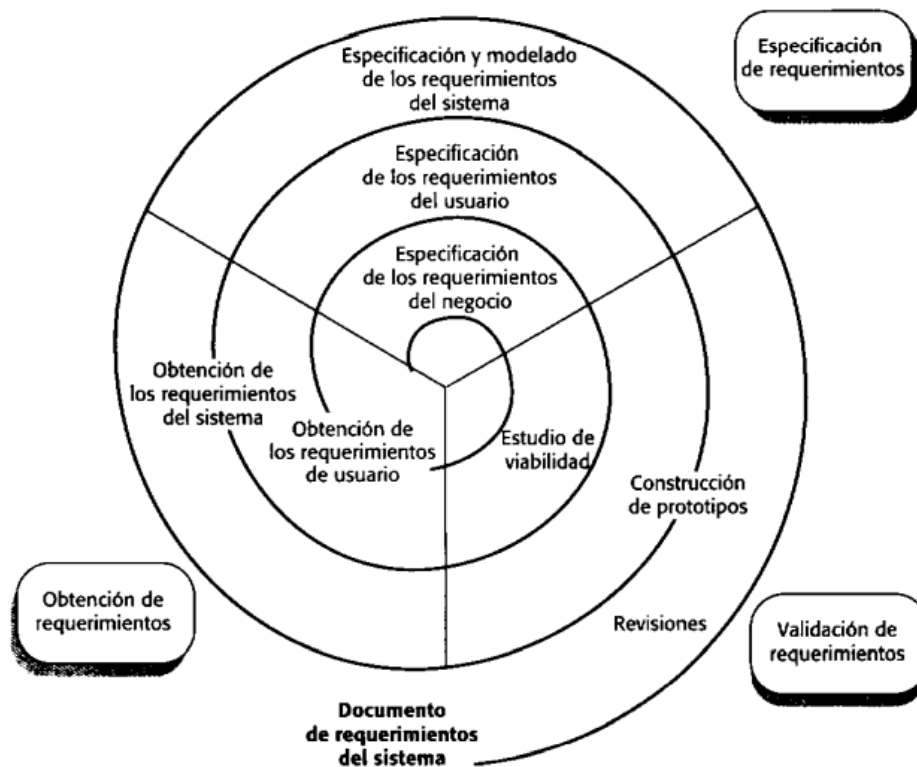


Ilustración 4: Modelo en espiral de los procesos de la ingeniería de requerimientos

Fuente:(Sommerville, 2005)

Este modelo en espiral responde a enfoques de desarrollo donde los requerimientos se desarrollan en diferentes niveles de detalle. El número de repeticiones alrededor de la espiral puede ser diferente, por lo tanto, se puede salir de la espiral después de tomar un poco de algunos o todos los requerimientos del usuario(Sommerville, 2005).

2. Modelados.

A nivel técnico, la ingeniería de software comienza con una serie de tareas de modelado, estos conducen a la identificación de requisitos y representa el diseño que se va a desarrollar.

El Modelo de requerimientos es un conjunto de modelos, es la primera representación técnica del sistema.

2.1. Modelado orientado al flujo

Aunque algunos ingenieros de software consideran que el modelo orientado al flujo es una técnica obsoleta, hasta el día de hoy sigue siendo uno de los símbolos más utilizados para realizar el análisis de requisitos. Aunque los diagramas de flujo de datos (DFD) y la información relacionada no forman parte de UML, se utilizan para completar diagramas UML y amplían la perspectiva de los requisitos del sistema y del flujo del sistema (Pressman, 2010).

El DFD aplica una perspectiva de entrada-proceso-salida al sistema; los objetos de datos ingresan al sistema y son convertidos por elementos de procesamiento y los objetos de datos resultantes abandonan el software. Los objetos de datos se representan con flechas con leyendas y transformaciones, con círculos (llamados burbujas). Los DFD se presenta en forma jerárquica. El primer modelo de flujo de datos (llamado DFD nivel 0 o diagrama de contexto) refiere a todo el sistema. Los diagramas de flujo de datos subsiguientes mejoran el diagrama de contexto y proporcionan más y más detalles en los siguientes niveles (Pressman, 2010).

2.1.1. Creación de un modelo de flujo de datos

Los diagramas de flujo de datos le permiten interpretar modelos para el campo de la información y lugares de trabajo, cuando el DFD se mejora con un mayor nivel de detalle es posible deterioro de la funcionalidad del sistema. Al mismo tiempo, la mejora de DFD aporta en el resultado de la mejora de los datos a medida que avanzan los procesos que los componen. Algunas pautas simples lo ayudarán al crear un diagrama de Flujo de datos: (Pressman, 2010)

- 1) El nivel 0 del diagrama debe mostrar el programa o sistema como una sola burbuja;
- 2) las entradas y salidas principales deben ser cuidadosamente observadas;

3) La optimización debe comenzar con el aislamiento de procesos candidatos y objetos de datos y almacenamiento para representarlos en el siguiente nivel;

4) Todas las flechas y burbujas deben etiquetarse con nombres Importancia;

5) De un nivel a otro, se debe conservar la continuidad del flujo de información, y

6) Debes mejorar una burbuja a la vez.

Existe un estilo natural a complicar innecesariamente el diagrama de flujo de datos. Esto ocurre cuando tratas de explicar demasiado detallado en una etapa muy temprana o representa los aspectos procedimentales del programa en el sitio del flujo información. Para ilustrar el uso de DFD y la codificación relacionada, considere la función de seguridad de CasaSegura(Pressman, 2010).

La ilustración 5 muestra el nivel 0 DFD para esta función, las entidades externas primarias (cuadrados) crean información diseñada para que la use el sistema y consumen la información que produce. Las flechas anotadas representan objetos de datos o jerarquía. Por ejemplo, los datos y comandos de usuario incluyen todos los comandos de configuración, todos los comandos de activación/desactivación, todos las distintas Interacciones y todos los datos ingresados para calificar o expandir un comando(Pressman, 2010).

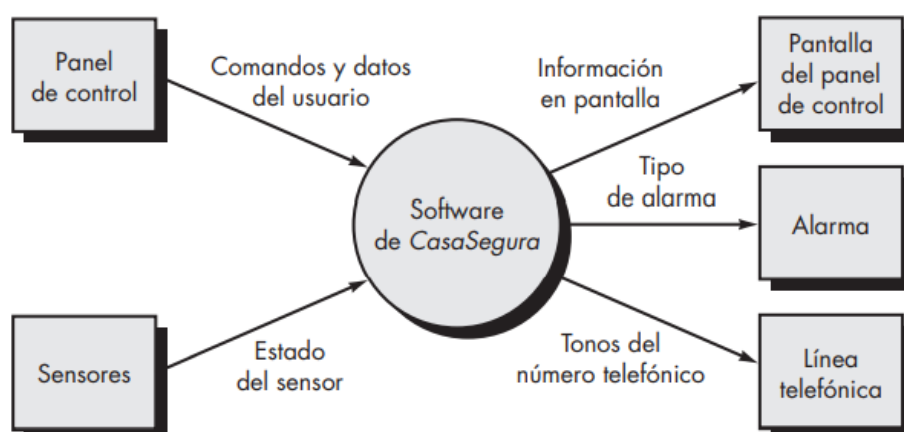


Ilustración 5: DFD en el nivel de contexto para la función de seguridad de CasaSegura
 Fuente: (Pressman, 2010)



Recuerde que. –El modelado del flujo de datos es una actividad fundamental del análisis estructurado(Pressman, 2010).

“La función de seguridad CasaSegura permite que el propietario configure el sistema de seguridad cuando se instala, vigila todos los sensores conectados al sistema de seguridad e interactúa con el propietario a través de internet, una PC o un panel de control. Durante la instalación, la PC de CasaSegura se utiliza para programar y configurar el sistema. Se asigna a cada sensor un número y un tipo, se programa una clave maestra para activar y desactivar el sistema, y se introducen números telefónicos para marcar cuando ocurre un evento de sensor. Cuando se reconoce un evento de sensor, el software invoca una alarma audible instalada en el sistema. Después de un tiempo de retraso que especifica el propietario durante las actividades de configuración del sistema, el software marca un número telefónico de un servicio de monitoreo, proporciona información acerca de la ubicación y reporta la naturaleza del evento detectado. El número telefónico vuelve a marcarse cada 20 segundos hasta que se obtiene la conexión telefónica. El propietario recibe información de seguridad a través de un panel de control, la PC o un navegador, lo que en conjunto se llama interfaz. La interfaz despliega en el panel de control, en la PC o en la ventana del navegador mensajes de aviso e información del estado del sistema. La interacción del propietario tiene la siguiente forma”

Fuente: (Pressman, 2010)

En lo que respecta al análisis gramatical, los verbos son operaciones de CasaSegura y se representarán como burbujas en DFD más adelante. Los sustantivos son entidades externas (cuadros), un dato, objetos de control (flecha) o almacén de datos (línea doble) (Pressman, 2010).



Recuerde que. – recordemos que los sustantivos y los verbos se combinan entre sí (por ejemplo, se asigna un número y tipo a cada sensor; Entonces número y tipo son atributos del objeto de datos sensor) (Pressman, 2010).

Por lo tanto, al realizar el procesamiento de análisis gramatical de la narración en cualquier nivel de DFD, se genera mucha información útil sobre cómo proceder para mejorar al siguiente nivel(Pressman, 2010).

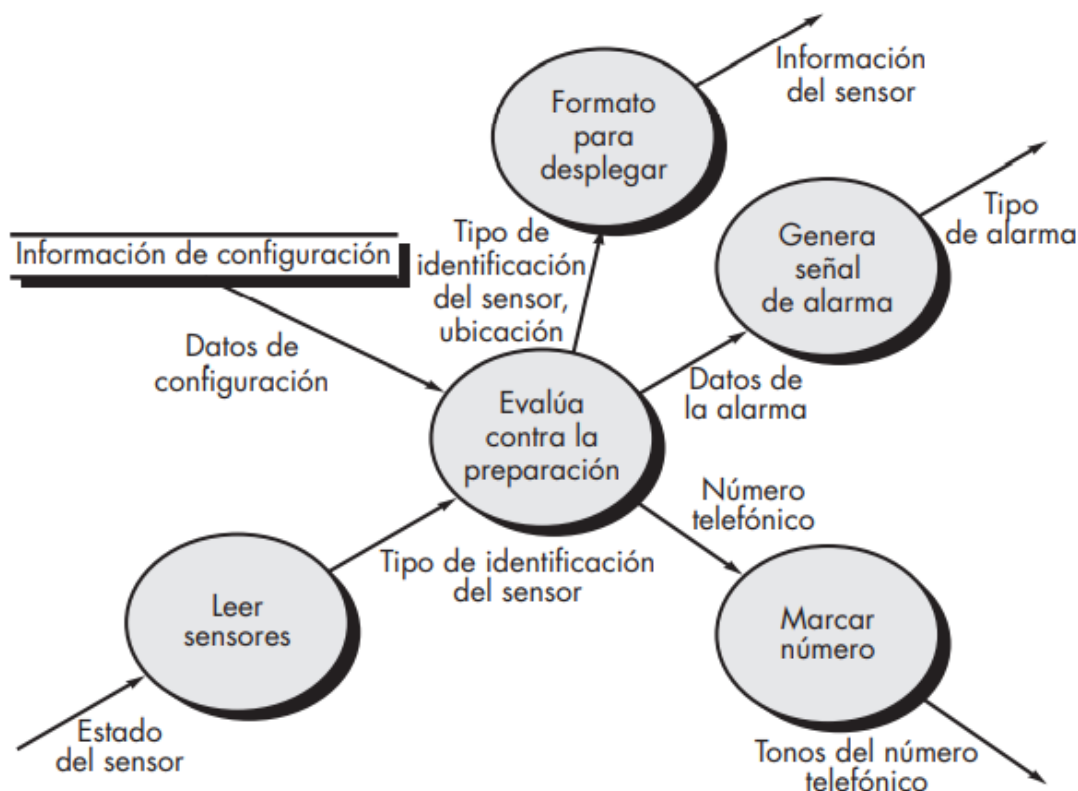


Ilustración 6: DFD de nivel 2 que mejora el proceso vigilar sensores
Fuente: (Pressman, 2010)

La ilustración 6 muestra un DFD de nivel 1 usando esta Información. El proceso a nivel de contexto que se muestra en la ilustración 5 se ha ampliado a seis procesos derivados del estudio de análisis gramatical. De manera similar, el flujo de información entre los procesos del Nivel 1 surge de dicho análisis. También, entre los niveles 0 y 1 se conserva el flujo de información. Los procesos representativos en el nivel DFD 1 se pueden mejorar aún más en los niveles inferiores. Por ejemplo, el funcionamiento del sensor de monitoreo mejorado en el nivel DFD 2 es como se muestra en la ilustración 7, una vez más, tenga en cuenta que la continuidad del flujo se ha mantenido entre los niveles(Pressman, 2010).

Las mejoras en DFD continuaron hasta que cada burbuja realizó una función simple. Eso equivale que el proceso representado por la burbuja implemente una función para ser publicado fácilmente como parte del programa(Pressman, 2010).

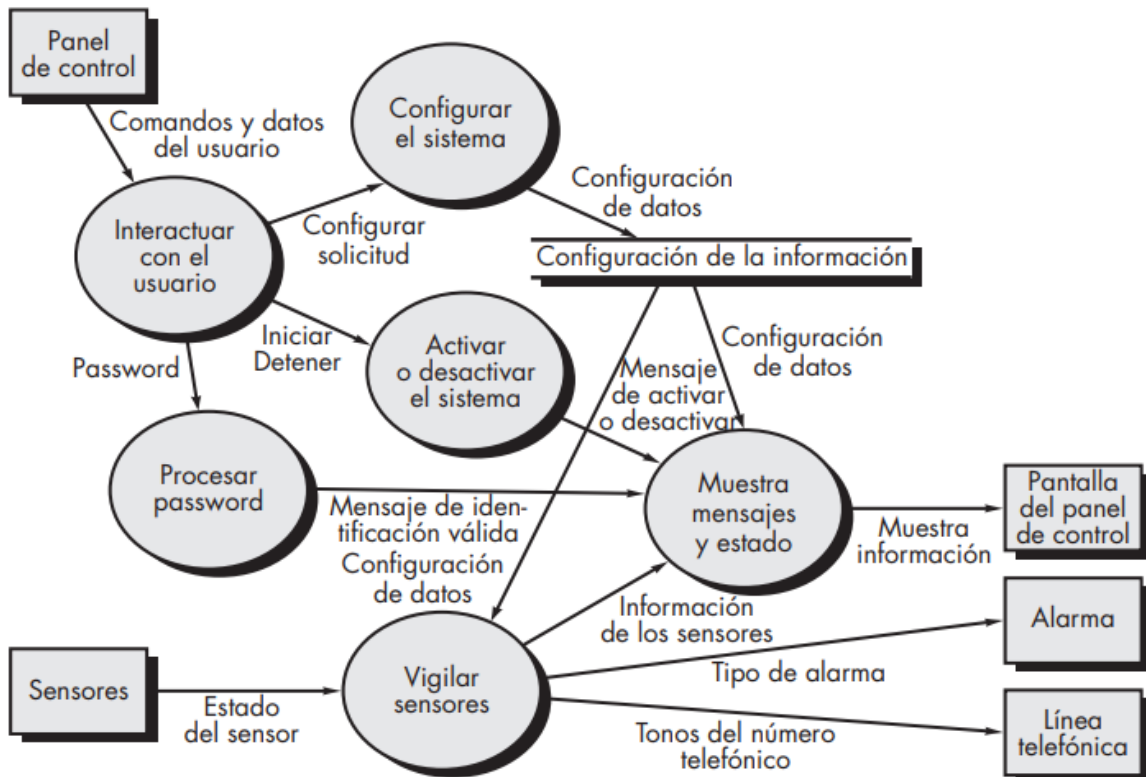


Ilustración 7: DFD de nivel 1 para la función de seguridad de CasaSegura
Fuente: (Pressman, 2010)

2.1.2. Creación de un modelo de flujo de control

Para algunos tipos de aplicaciones, los modelos de datos y los diagramas de flujo de datos son suficientes necesarios para obtener una visión significativa de los requisitos del software. Sin embargo, como dijimos, una gran cantidad de aplicaciones son "impulsadas" por eventos y no por los datos, crean información de control en lugar de informes o pantallas, y procesan la información con gran atención al tiempo y al rendimiento. Estas aplicaciones requieren el uso del Modelo de flujo de control, al mismo tiempo del modelo de flujo de datos. Se ha dicho que un evento o aspecto de un

control se ejecuta como un valor booleano (por ejemplo: verdadero o falso, encendido o apagado, 1 o 0) o como una lista separada de condiciones (vacío, prohibido, bloqueado, completo, etc.). Se aconsejan las siguientes pautas para verificar los hechos de eventos candidatos potenciales: (Pressman, 2010)

Listar todos los sensores "leídos" por el software.

- Enumere todos los términos del boicot.
- Listar todos los "interruptores" que han sido activados por el operador.
- Una lista de todas las condiciones de datos.
- Considerar todos los "aspectos de control" como posibles entradas o salidas a la especificación de control, con base en el análisis gramatical de sustantivos y verbos aplicados a procesamiento de la narración.
- Describir el comportamiento del sistema definiendo sus estados, determinando cómo llegar a cada estado e identificando transiciones entre estados.
- Centrarse en posibles omisiones, que es un error muy común al definir controles; porque, Por ejemplo, debe preguntarse: "¿Hay otra forma de entrar o salir de este estado?"

Entre los muchos eventos y aspectos de control que forman parte del programa CasaSegura, se encuentran los eventos del sensor (como la falla del sensor), las señales de advertencia, bandera de Cambio (indicación en pantalla para el cambio) e interruptor de encendido/apagado (indicación para encender o apagar el sistema)(Pressman, 2010).

2.1.3. La especificación de control

La especificación de control (CSPEC) muestra el comportamiento de dos maneras diferentes. CSPEC contiene un diagrama de estado que es un determinante secuencial de la conducta. También puede contener una tabla para la activación del programa, una especificación combinatoria de comportamiento(Pressman, 2010).

La ilustración 8, muestra el diagrama de estado inicial para el nivel 1 del modelo de flujo de control de CasaSegura. El diagrama muestra cómo reacciona el sistema a los eventos a medida que pasan por los cuatro países identificados en este nivel. La prueba del mapa de estado identifica el comportamiento del sistema y, lo que es más importante, busca cualquier "agujero" en el comportamiento especificado. Por ejemplo, el diagrama de estado (ver ilustración 8) muestra que las transiciones de estado ocioso suceden si el sistema se reinicia, enciende o apaga.

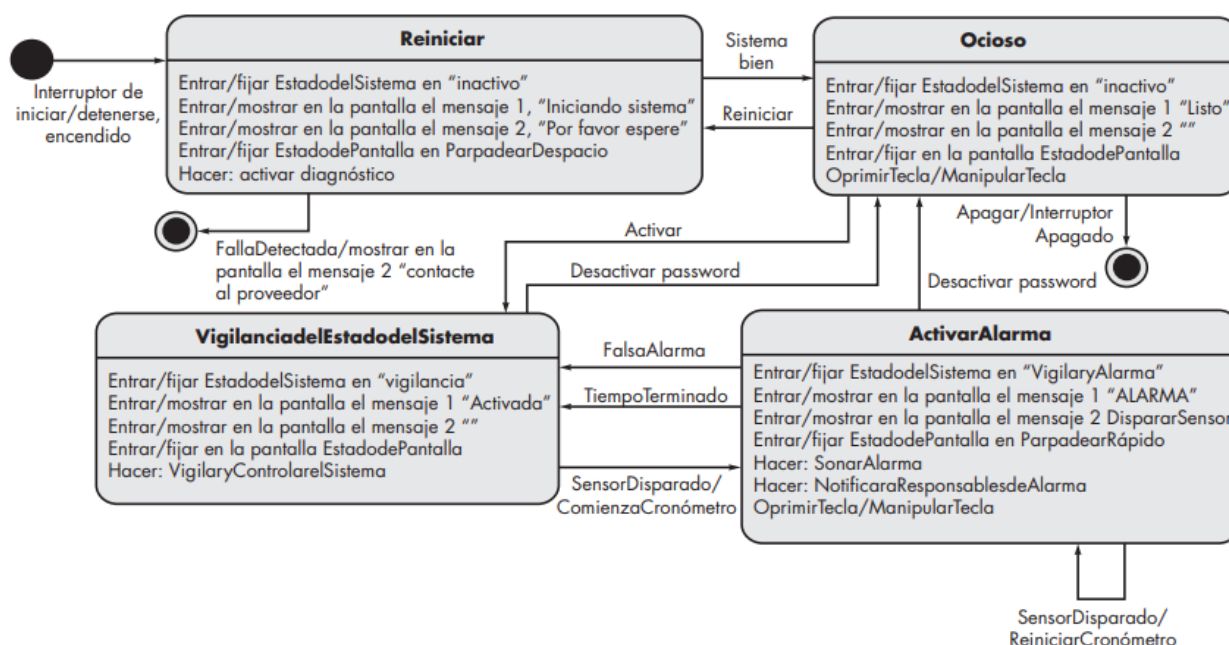


Ilustración 8: Diagrama de estado para la función de seguridad de CasaSegura
Fuente: (Pressman, 2010)

Si el sistema se está ejecutando (por ejemplo, se activa un sistema de alarma), se produce una transición al estado Vigilancia del Estado del Sistema, el mensaje en pantalla cambia como se muestra y se llama al proceso Sistema de Vigilancia y Control (Sistema de seguimiento y control). Fuera del estado del sistema de monitoreo y control, ocurren dos transiciones: 1) cuando el sistema está caído, hay una transición al estado inactivo; 2) cuando se activa el sensor, se activa la alarma. Durante la inspección se considera todas las transiciones y contenidos de todos los estados. La tabla de activación de procesos (TAP) es una forma ligeramente diferente de

expresar el comportamiento. TAP representa la información en el diagrama de estado en contexto del proceso, no del estado. Es decir, la tabla muestra el proceso (burbuja) serán llamados en el modelo de asociación cuando ocurre un evento. TAP se usa como directorio para un Diseñador, que necesitan construir una aplicación que controle los procesos descritos en este nivel(Pressman, 2010).

<u>eventos de entrada</u>						
evento de sensor	0	0	0	0	1	0
bandera de parpadeo	0	0	1	1	0	0
interruptor de iniciar o detener	0	1	0	0	0	0
estado de la acción en pantalla terminado	0	0	0	1	0	0
en marcha	0	0	1	0	0	0
tiempo terminado	0	0	0	0	0	1
<u>salida</u>						
señal de alarma	0	0	0	0	1	0
<u>activación del proceso</u>						
vigilar y controlar el sistema	0	1	0	0	1	1
activar o desactivar el sistema	0	1	0	0	0	0
mostrar mensajes y estado	1	0	1	1	1	1
interactuar con el usuario	1	0	0	1	0	1

Ilustración 9: Activación del proceso para la función de seguridad de CasaSegura
 Fuente: (Pressman, 2010)

La ilustración 9 muestra el TAP para el Nivel 1 del modelo de flujo del programa para la CasaSegura. CSPEC describe el comportamiento del sistema, pero no proporciona información sobre el funcionamiento interno de los procesos que desencadena este comportamiento(Pressman, 2010).

2.1.4. La especificación del proceso

La especificación de proceso (PSPEC) se utiliza para describir todos los procesos en un modelo que aparece en el nivel final de la mejora. El contenido de la operación incluye texto narrativo y una descripción del lenguaje de diseño del algoritmo del proceso, ecuación matemática, tabla o diagrama de una actividad UML. Si se proporciona un PSPEC para acompañar cada

burbuja en el modelo de flujo, se genera un 'miniespecificación' para la distribución como guía, para diseñar el componente de software que disparará la burbuja. Para ilustrar el uso de PSPEC, considere la transformación procesar password presentada En el modelo de flujo de la ilustración 7. PSPEC para esta función tiene la siguiente forma:(Pressman, 2010).

PSPEC: procesar password (en el panel de control). La transformación procesar password realiza la validación en el panel de control para la función de seguridad de CasaSegura. Manejo de contraseñas obtiene una contraseña de cuatro dígitos de la función de interacción del usuario. Primero, la contraseña es comparada con la contraseña maestra almacenada en el sistema. Si la contraseña maestra coincide, se pasa para mostrar el mensaje y la función de estado (pasa <mensaje de identificación válida = verdadero> a la función mostrar mensaje y estado), Si la contraseña maestra no coincide, los cuatro dígitos se comparan con una serie de contraseñas secundarias (deben especificarse para acomodar a los invitados o trabajadores que necesitan ingresar a la casa cuando el propietario no esté presente), Si la contraseña coincide con una entrada en la tabla, <mensaje de identificación válida = verdadero>para mostrar la función de notificación y el estado. Si no coinciden, ignorar <mensaje de identificación válida = falso> a la función de mostrar mensaje y estado. Si se requieren detalles algorítmicos adicionales en este punto, también se puede incluir una representación de lenguaje de diseño de programa (PDL) en PSPEC. pero, muchos expertos creen que el lanzamiento del LDP debería retrasarse hasta el inicio del diseño de los componentes(Pressman, 2010).

2.2. Modelo de flujo de datos

Un modelo de flujo de datos es una forma visual de mostrar cómo el sistema maneja los datos. En el nivel analítico, debe usarse para modelar los datos, estos se procesan en el sistema actual. Estos modelos son una parte integral de a partir de este trabajo se desarrollaron métodos estructurados. La notación representa donde se almacena la función de procesamiento (rectángulo redondeado), los almacenes de datos (el rectángulo) y la cantidad de datos entre las funciones (flechas etiquetadas). El modelo de flujo de datos

se utiliza para mostrar cómo fluyen los datos por medio de una serie de pasos de procesamiento. Por ejemplo, el paso de procesamiento puede ser filtrar los registros duplicados en la base de datos de clientes. Los datos se transfieren en cada paso antes de pasar al siguiente paso. Estos pasos de procesamiento o conversión representan procesos o funciones de software utilizando diagramas de flujo de datos. Que sirven para documentar el diseño del programa. Sin embargo, el modelo analítico, puede ser realizado por un humano o por una computadora(Sommerville, 2005).

Modelo de flujo de datos, muestra los pasos involucrados en el procesamiento de datos de los pedidos de productos (como equipamiento informático) en una organización se muestran en la ilustración 10(Sommerville, 2005).

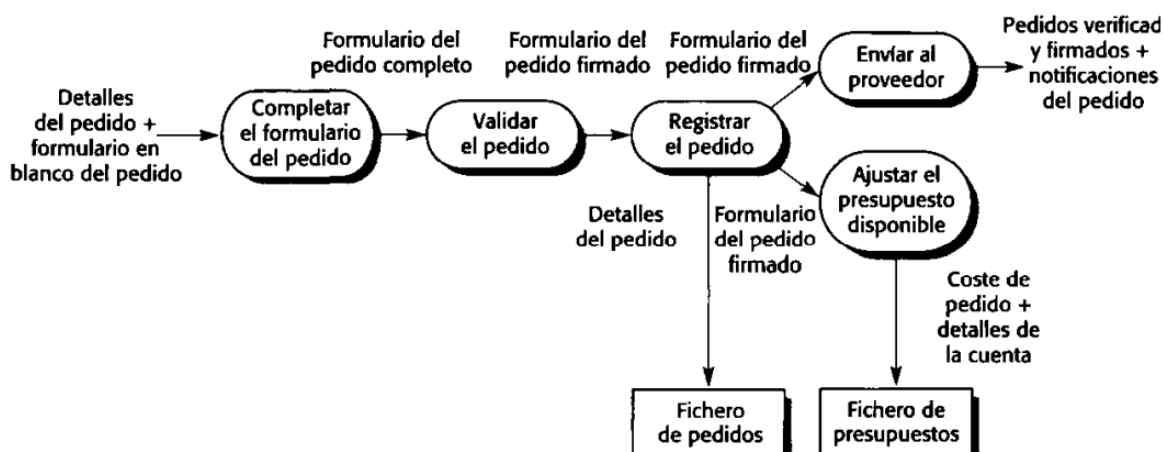


Ilustración 10: Diagrama de flujo de datos del procesamiento de un pedido.

Fuente: (Sommerville, 2005)

Este modelo particular describe el procesamiento de datos en una actividad. Establezca la disposición del dispositivo en el modelo de proceso completo como se muestra en la ilustración 10. El modelo muestra cómo los pedidos de productos se mueven de un proceso a otro. También muestra almacenes de datos (archivos de solicitud y archivos de cotización) de las personas que participaron en este proceso. Los modelos de flujo de datos son valiosos porque rastrean y documentan cómo los datos relacionados con un proceso en particular fluyen a través del sistema y ayudan a los analistas a

entender el proceso. Los diagramas de flujo de datos, a diferencia de otros símbolos modelo, son simples e intuitivos. A menudo, es posible explicárselo a los usuarios potenciales del sistema, quienes pueden participar en la validación del análisis(Sommerville, 2005).

En principio, los modelos como el modelo de flujo de datos deben desarrollarse en proceso de arriba hacia abajo. En este ejemplo, podría significar que debe comenzar a analizar todo el proceso de adquisición de equipos. Entonces se seguiría analizando temas como solicitudes de pedidos. De hecho, el análisis nunca se hace de esta manera, se analizan varios niveles al mismo tiempo(Sommerville, 2005).

Los modelos de bajo nivel se pueden expandir primero y luego extraer para crear un modelo más complejo. Los modelos de flujo de datos muestran una vista funcional en la que cada transformación representa un proceso o función. Es particularmente útil durante el análisis de requisitos porque se puede utilizar para visualizar el procesamiento de extremo a extremo en un sistema. Es decir, muestra toda la secuencia de acciones que dan la posición de la entrada procesada en la salida correspondiente constituye la respuesta del sistema(Sommerville, 2005).

2.3. Modelado de datos

La mayoría de los notables sistemas de software utilizan grandes bases de datos de información. En algunos casos, esta base de datos es independiente del sistema del programa, en otros, fue creado para el sistema en desarrollo. Una parte importante del modelado de sistemas es definir la forma lógica de los datos que el sistema procesará. A menudo se les conoce como modelos de datos semánticos. La técnica de modelado de datos más utilizada es el modelado Entidad-Relación-Atributo (modelado ERA), que presenta entidades de datos, sus atributos asociados y las relaciones entre esas entidades. Este enfoque de modelado fue propuesto por primera vez a mediados de los setenta por Chen (Chen, 1976); desde entonces, se han desarrollado varios cambios (Codd, 1979; Hammer y McLeod, 1981; Hull y King, 1987), todas con la misma forma básica(Sommerville, 2005).

Los modelos de entidad-relación se han utilizado ampliamente en el diseño de bases de datos. Los esquemas de bases de datos relacionales procedentes de estos modelos se hallan normal en la tercera forma normal, que es una propiedad deseable (Barker, 1989). Debido a la forma en que se importan y reconocen explícitamente los subtipos y supertipos, también es fácil de implementar estos modelos utilizando bases de datos orientadas a objetos (Sommerville, 2005).

UML no incluye una notación específica de este modelo de base de datos porque asume un proceso de modelado de datos y desarrollo orientado a objetos utilizando objetos y sus relaciones (Sommerville, 2005).

Sin embargo, UML se puede utilizar para representar un modelo semántico de datos, puede pensar en entidades en un modelo ERA como clases de objetos simplificadas (no con operaciones); atributos que son atributos de clases y las llamadas asociaciones entre clases como relaciones (Sommerville, 2005).

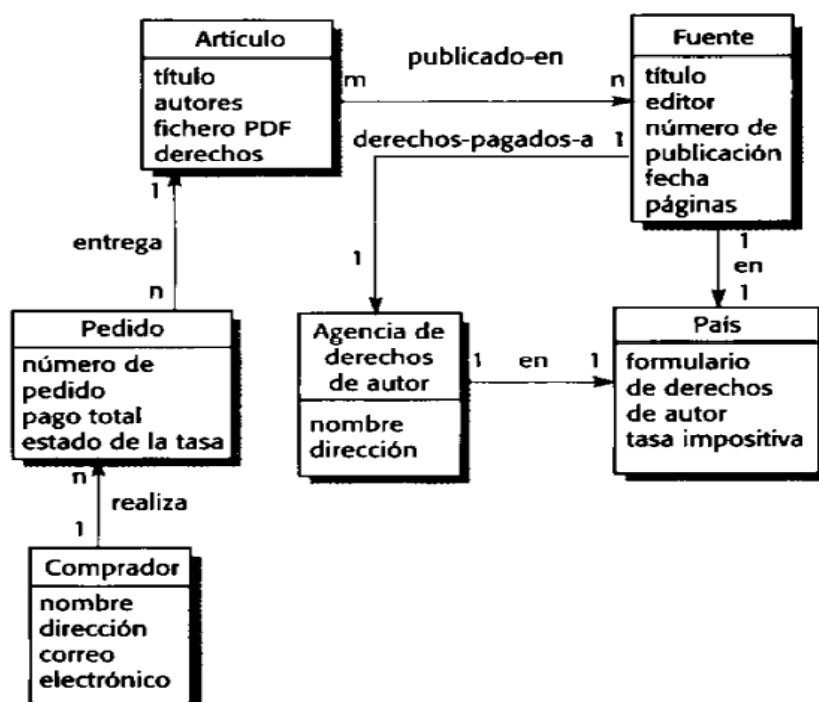


Ilustración 11: Modelo semántico de datos para el sistema L1BSYS
 Fuente: (Sommerville, 2005)

La ilustración 11 es un ejemplo de un modelo de datos que forma parte de un sistema de biblioteca LIBSYS, Recuerde que LIBSYS está diseñado para brindar copias de artículos protegidos por derechos de autor publicados en revistas, periódicos y para registrar el pago de estos artículos. Por lo tanto, el modelo de datos debe contener información sobre el artículo, el titular de los derechos de autor y la persona que compró el artículo. Hemos asumido aquí que estos pagos por artículos no se realizan directamente sino a través de las agencias nacionales de derechos de autor. La ilustración 11 muestra un artículo con propiedades que representan el título, el autor y el nombre en PDF del artículo y las tarifas a pagar. Esto indica la fuente, donde se publicó el artículo y la oficina de derechos de autor en el país de publicación, Copyright y Agencia de derechos de autor y Fuente, se enlazan con País. El país de publicación es importante porque las leyes de derechos de autor varían de un país a otro. El diagrama también muestra que los compradores hacen pedidos de publicaciones(Sommerville, 2005).

Como todos los modelos gráficos, los modelos de datos carecen de detalles y se debe mantener descripciones más detalladas de las entidades, relaciones y atributos en el modelo, se puede recopilar estas descripciones más detalladas en un almacén de datos o diccionario. Los diccionarios de datos suelen ser útiles cuando se desarrolla un modelo de sistema y se pueden utilizar para gestionar toda la información del sistema y tipo de modelos de sistemas(Sommerville, 2005).

Un diccionario de datos es simplemente una lista alfabética de nombres incluido en el modelo del sistema. Además, el diccionario debe incluir una descripción asociada con la entidad dada, y si el nombre representa un objeto compuesto, se debe describir el elemento. Se puede incluir otra información, como la fecha de creación, creador y representación de la entidad según el tipo de modelo que se esté desarrollando(Sommerville, 2005).

Los beneficios de usar un diccionario de datos son:

- Es un mecanismo de gestión de nombres. Muchas personas pueden tener que inventar nombres para entidades y relaciones al

desarrollar un modelo para un sistema grande. Estos nombres deben usarse de manera consistente y no debe entrar en conflicto. Software de diccionario de datos puede comprobar la unicidad del nombre si es necesario y asesorar a los analistas sobre reclamos de nombres duplicados(Sommerville, 2005).

- Sine como un almacén de información de la organización: a medida que el sistema es desarrollado, analizado, diseñado, implementado y evaluación se añade la información en el diccionario de datos, para que toda la información este sobre la entidad este en el mismo lugar(Sommerville, 2005).

Las entradas del diccionario de datos que se muestran en la ilustración 12 especifican nombres en el modelo de datos semánticos de LIBSYS(Sommerville, 2005).

Artículo	Detalle del artículo publicado que puede pedirse por las personas que usen LIBSYS	Entidad	30.12.2002
Autores	Los nombres de los autores del artículo que pueden compartir los honorarios	Atributo	30.12.2002
Comprador	La persona u organización que emite una orden de copia del artículo	Entidad	30.12.2002
Honorarios a pagar a	Una relación 1:1 entre Artículo y la Agencia de derechos de autor a quien se debería abonar el honorario de derechos de autor	Relación	29.12.2002
Dirección (Comprador)	La dirección del comprador. Ésta se utiliza para cualquier información que se requiera sobre la factura en papel	Atributo	31.12.2002

Ilustración 12: Ejemplos de entradas de diccionario de datos.
Fuente: (Sommerville, 2005)

REFERENCIA BIBLIOGRÁFICA

- Pressman, R. S. (2010). *Ingeniería del Software Un enfoque práctico* (Séptima Ed). McGraw-Hill. <http://cotana.informatica.edu.bo/downloads/Id-Ingenieria.de.software.enfoque.practico.7ed.Pressman.PDF>
- Sommerville, J. A. N. (2005). *Ingeniería del software Ingeniería del software Séptima edición*.